📘 Playwright Automation - Assignment 6

Total Questions: 25
Theory Questions: 20
Programming Questions: 5

Section A – Theory Questions

Q1. What is Playwright? Explain how Playwright is different from Selenium. Mention at least four practical advantages of Playwright that make it a preferred choice for modern web application testing.

Q2. Explain the Playwright architecture. Which protocol does Playwright use to communicate with browsers? Why is it faster and more reliable than the WebDriver protocol?

Q3. Explain the difference between Browser, BrowserContext, and Page in Playwright. In a real project, when would you create multiple BrowserContext instances instead of multiple pages?

Q4. What is Auto Waiting in Playwright? Explain at least five actionability checks performed automatically before interacting with an element. How does Auto Waiting reduce flaky tests?

Q5. What is Codegen in Playwright? Should we completely depend on Codegen-generated scripts in a production framework? Justify your answer.

Q6. Explain the difference between Hard Assertions and Soft Assertions. Give one real-time scenario where Soft Assertions are more useful than Hard Assertions.

Q7. What is the purpose of the playwright.config.js file? Explain any six important configuration options used in real-time automation projects.

Example:

- retries
- workers
- baseURL
- use
- projects
- reporter

Q8. Explain the difference between:

- page.click()
- page.locator().click()

Which approach is recommended in modern Playwright projects? Why?

Q9. Differentiate between:

- page.fill()
- page.type()

In which scenario would page.type() be preferred over page.fill()?

Q10. Explain the following Playwright methods with one real-time use case for each.

- page.reload()
- page.goBack()
- page.goForward()
- page.waitForTimeout()

Also explain why waitForTimeout() should be avoided in production automation frameworks.

Q11. Arrange the following locator strategies in the recommended Playwright priority order and explain why.

- getByRole()
- getByLabel()
- getByText()
- getByPlaceholder()
- getByTestId()
- CSS Selector
- XPath

Q12. Differentiate between:

- getByText()
- getByLabel()

Explain with real-time UI examples where each locator should be used.

Q13. What is getByTestId()? Why is it considered one of the most stable locator strategies in enterprise applications?

Q14. Explain the difference between:

- Static Dropdown
- Dynamic Dropdown

Which Playwright method is used for handling static dropdowns? Why doesn't it work for custom dropdowns built using React or Angular?

Q15. How do you handle a Multi-Select Dropdown in Playwright? Explain using a real-time example.

Q16. Explain the different types of screenshots supported by Playwright.

- Full Page Screenshot
- Element Screenshot

Mention one real-time use case for each.

Q17. What is an iframe? Why can't Playwright identify iframe elements using page.locator()? How does frameLocator() solve this problem?

Q18.How do you handle nested iframes in Playwright?
Write only the locator chain (not the complete program) for the following structure:
Parent iframe → Child iframe → Input field with placeholder "Enter OTP"

Q19. Differentiate between the following popup types.

- JavaScript Alert
- JavaScript Confirm
- JavaScript Prompt
- Authentication Popup
- Permission Popup

Also explain which popups are handled using:
page.on("dialog")
and which are handled using Browser Context.

Q20. Why should page.on("dialog") always be registered before clicking the button that triggers the alert?
Explain what happens if the listener is registered after the alert appears.

Section B – Programming Questions

Q21. Write a Playwright program that:

1. Opens https://the-internet.herokuapp.com/javascript_alerts
2. Clicks the JavaScript Alert button.
3. Accepts the alert.
4. Validates the success message displayed on the page.

Q22. Write a Playwright program that:

1. Opens https://the-internet.herokuapp.com/javascript_alerts
2. Clicks the JavaScript Prompt button.
3. Enters the text "Playwright Automation"

4. Accepts the prompt.
5. Validates the entered text displayed on the page.

Q23. Write a Playwright program that:

1. Opens https://playwright.dev
2. Captures a Full Page Screenshot named:

playwright-home.png

3. Captures an Element Screenshot of the top navigation bar named:
   nav-bar.png

4. Validates that the page title contains Playwright.

Q24. Write a Playwright program that:

1. Opens https://the-internet.herokuapp.com/iframe
2. Switches to the iframe.
3. Clears the existing text.
4. Types:

Automation using Playwright iframe handling

5. Validates the entered text.

Q25. Write a Playwright program that:

1. Opens https://the-internet.herokuapp.com/dropdown
2. Selects Option 2 using selectOption().
3. Validates:
   - Selected value is 2
   - Selected visible text is Option 2
4. Prints all dropdown options to the console.